Téléversement : Mise à jour des photos des étudiants

Dans le cadre de la gestion des étudiants, il faut maintenant prendre en compte la gestion de leur photo.

La première mission consiste à revenir sur la suppression d'un étudiant qui doit entrainer la suppression de la photo associée.

La seconde mission consiste à compléter le nouveau module 'majphoto' permettant d'ajouter, de modifier ou de supprimer la photo des étudiants.

La troisième mission consiste à proposer une nouvelle version du module 'ajout' permettant d'intégrer la photo de l'étudiant, cette dernière restant cependant facultative.

1. Eléments ajoutés au projet pour permettre la gestion des photos

Les classes techniques suivantes fournissent les briques de validation et de stockage des photos :

- **Input** (src/ClasseTechnique/Input.php) est la classe de base des objets de saisie. Elle gère une valeur, son caractère obligatoire et les messages de validation.
- **InputFile** (src/ClasseTechnique/InputFile.php) hérite de Input et représente un fichier téléversé. Elle contrôle notamment l'erreur de téléversement, la taille, l'extension, le type MIME et le nom du fichier.
- **InputFileImg** (src/ClasseTechnique/InputFileImg.php) hérite de InputFile et ajoute les contrôles propres aux images, comme la validité du contenu et les dimensions.
- **FileManager** (src/ClasseTechnique/FileManager.php) centralise les opérations génériques sur les fichiers : vérifier leur nom, les ajouter, les remplacer, vérifier leur présence et les supprimer.
- **ImageManager** (src/ClasseTechnique/ImageManager.php) hérite de FileManager et spécialise le stockage pour les images, notamment par la validation des formats et le redimensionnement configuré.

Le projet utilise également deux bibliothèques Composer :

- **gumlet/php-image-resize (^3.0)** est la bibliothèque appelée par ImageManager pour redimensionner les images tout en conservant leurs proportions. Son intégration est encapsulée dans la classe technique ; le service n'a qu'à fournir les options de configuration.
- **ezyang/htmlpurifier (^4.19)** nettoie et sécurise les contenus HTML afin d'empêcher l'injection de balises ou de scripts indésirables. Il est utilisé par ClasseTechnique\Std pour les traitements HTML et n'intervient pas directement dans l'enregistrement ou la suppression des photos.

Les paramètre de configuration concernant les photos seront stockés dans le fichier config/etudiant.php
Le fichier bootstrap.php contient maintenant la constante indiquant le répertoire de stockage des photos

```
// constantes.php pour les dossiers comprenant des resources externes
const DOSSIER_PHOTO_ETUDIANT = DOSSIER_WWW . '/data/photo/';
```

Remarque : Actuellement, il existe une constante identique dans la classe Etudiant qui est utilisée dans les méthodes getAll(), getById() et getByNomPrenom(). Normalement cette constante devrait disparaitre

Important : Vous devez relancer le script sql/etudiant.sql pour avoir des données cohérentes entre la table etudiant et le répertoire public/data contenant les photos associées aux enregistrements de la table etudiant

2. Suppression de la photo lors de la suppression d'un étudiant

La suppression complète d'un étudiant touche donc maintenant deux responsabilités différentes :
1. supprimer l'enregistrement étudiant de la base de données ;
2. effacer la photo associée du répertoire de stockage.

Dans une approche théorique, pour gérer les deux responsabilités, il faudrait passer par la mise en place d'une couche Service.

La couche Service se place entre le contrôleur et les classes métier / techniques pour coordonner ce cas d'utilisation. ServiceEtudiant conserve la logique d'orchestration dans une seule classe réutilisable, tandis que :

- le contrôleur reste centré sur HTTP et JSON ;
- Etudiant reste responsable des données et règles métier de l'étudiant ;

Dans une approche plus pragmatique et donc plus simple, plus compréhensible, c'est au contrôleur "public/suppression/ajax/supprimer.php" de prendre en charge cette tâche.

Puisqu'il appelle déjà la méthode "$etudiant->delete($id);" pour supprimer l'étudiant dans la table étudiant, il va enchainer par la suppression de la photo en utilisant la classe FileManager.

Remarque : nous utilisons la classe FileManager, bien que le fichier soit une image, car nous n'avons besoin que de supprimer le fichier en appelant la méthode

Le fichier "config/etudiant.php" doit contenir les paramètres appliqués sur les photos :
- La photo ne doit pas dépasser 150 Ko,
- Elle sera redimensionnée dans le format 150 * 150.
- Elle sera refusée en cas de doublon
- Les extensions 'jpg' et 'png' sont les seules extensions autorisées
- Le répertoire de stockage sera obtenu à partir de la constante *DOSSIER_PHOTO_ETUDIANT*

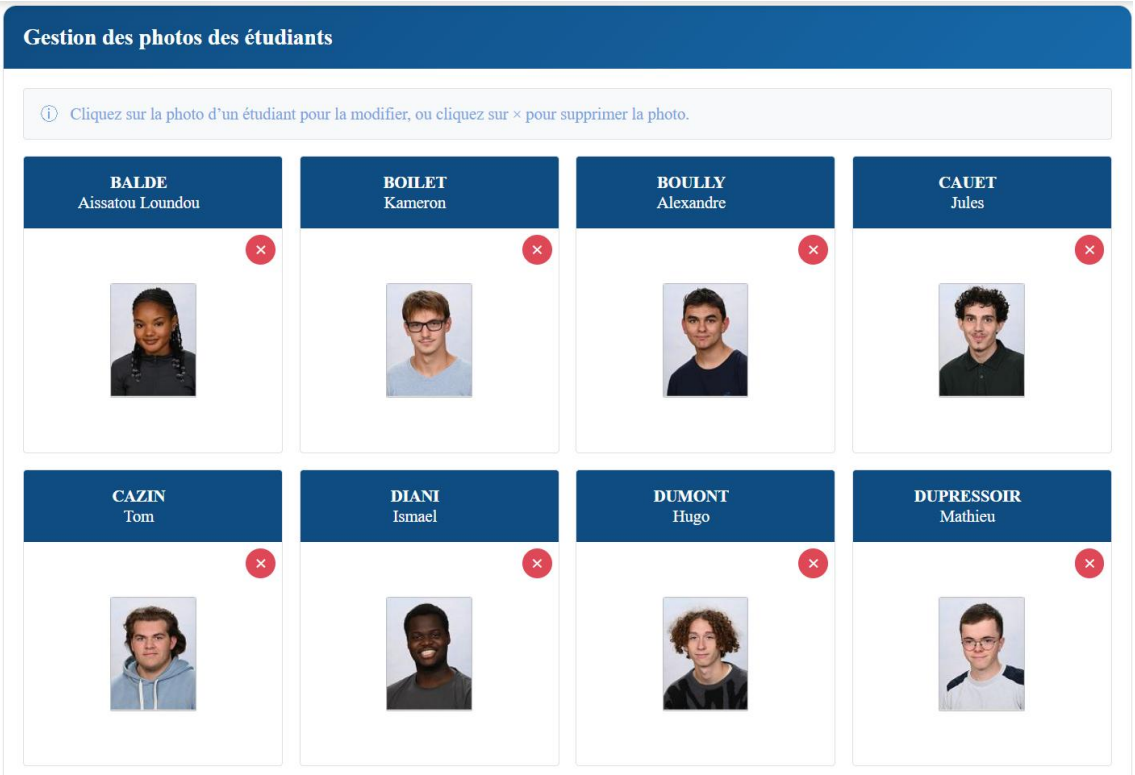
1. Compléter le fichier de configuration "config/etudiant.php".

2. Compléter le contrôleur "public/suppression/ajax/supprimer.php"

3. Vérifier le fonctionnement du module de suppression.

3. Mise à jour de la photo

La mise à jour des photos s'effectue à partir de l'interface suivante.



L'application doit permettre de :

- afficher une carte pour chaque étudiant ;
- afficher sa photo ou, en son absence, `public/data/photo/0.png` ;
- sélectionner une image en cliquant sur la photo ou la déposer par glisser-déposer ;
- vérifier le fichier côté navigateur, puis le valider à nouveau côté serveur ;
- réduire les images aux dimensions prévues dans le fichier de configuration ;
- remplacer une photo en conservant son nom de fichier existant ;
- enregistrer la première photo sous la forme `NOM_prenom.ext` et mémoriser ce nom dans la base de données ;
- supprimer uniquement la photo d'un étudiant, sans supprimer l'étudiant ;
- réafficher immédiatement l'image par défaut après la suppression ;
- afficher une confirmation ou une erreur à l'utilisateur.

Fonctionnement attendu

Affichage initial

Le contrôleur de page charge les étudiants et les paramètres concernant les photos. Ces paramètres sont stockés dans le fichier de configuration.
Le contrôleur transmet ces données côté client.
InterfaceHtml injecte les données dans une balise JSON et charge les fichiers HTML, CSS et JavaScript du module.
Le JavaScript lit les données, clone le template HTML et construit une carte par étudiant.
Une photo réellement présente est affichée. Sinon, la carte utilise 0.png et masque son bouton de suppression.

Ajout ou remplacement de la photo

L'utilisateur clique sur une photo pour ouvrir l'explorateur permettant de sélectionner une photo.
Le JavaScript valide l'extension et la taille, puis vérifie que le fichier est une image.
Une requête POST en FormData envoie le fichier et l'identifiant étudiant au contrôleur AJAX.
Le contrôleur :

- Lit les paramètres avec les méthodes de la classe Requete,
- Vérifie la transmission de la photo et de l'identifiant
- Vérifie l'existence de l'étudiant et en profite pour récupérer la valeur du champ photo.
- Vérifie la validité de la photo transmise
- Génère le nom de la nouvelle photo
- Copie cette photo dans le répertoire de stockage
- Met à jour le champ photo dans la table si le nom de la photo a changé
- Supprime l'ancienne photo s'il y en avait une

Attention : en cas d'échec lors de la mise à jour dans la table, il faut revenir à l'état précédent en supprimant la nouvelle photo.
L'échec peut être détectée par la méthode modify de la classe Table qui dans ce cas retourne false et permet de récupérer les erreurs à l'aide de sa méthode getErrors
L'échec peut être déclenchée dans la méthode modify, suite à une exception Pdo. Il faut dans ce cas capturer l'exception afin de pouvoir supprimer la nouvelle photo puis relancer l'exception afin qu'elle soit comme les autres traitée par le gestionnaire d'erreur (la classe Erreur)
Le serveur renvoie une réponse au format JSON contenant le nom donné à la photo afin que côté client elle puisse être affichée.

Le traitement de la réponse par la fonction appelAjax() consiste à afficher la photo dans la zone cible attaché à l'étudiant.

Suppression de la photo

Le bouton de suppression n'est disponible que si une photo existe.
Le JavaScript envoie l'identifiant étudiant au contrôleur AJAX de suppression.
Le contrôleur :

- Lit le paramètre primaryKey avec Requete,
- Vérifie l'existence de l'étudiant et en profite pour récupérer la valeur du champ photo.
- En l'absence de photo : ReponseJson::*envoyerErreur*("L'étudiant n'a pas de photo à supprimer.");
- Créer un objet métier 'Etudiant' pour mettre à jour le champ photo (valeur Null)
- Attention : La modification de la colonne 'photo' doit être autorisée
- Renvoie une réponse de réussite pouvant prendre trois valeurs différentes :
- ReponseJson::*envoyerMessage*("La photo de l'étudiant a été supprimée de la base de données, mais le fichier n'a pas pu être retiré du disque.");
- ReponseJson::*envoyerMessage*("La photo de l'étudiant a été supprimée de la base de données, mais le fichier n'existe pas sur le disque.");
- ReponseJson::*envoyerMessage*("La photo de l'étudiant a été supprimée de la base de données et du disque.");

Le traitement de la réponse par la fonction appelAjax() consiste à :

- masquer l'image,
- afficher la zone permettant d'ajouter une photo,
- masquer le bouton de suppression,
- et afficher le message renvoyé par le serveur

Téléversement : Mise à jour des photos des étudiants

4. Compléter le fichier photo/index.php

Dans public/photo/index.html, le champ de fichier est caché et le template définit une carte par étudiant. L'image de la carte déclenche la sélection d'un fichier ; le bouton de suppression est affiché uniquement lorsqu'une photo existe.
La suppression est traitée par un autre endpoint (supprimerphoto.php), distinct de la mise à jour.

Le JavaScript public/photo/index.js :

- crée une carte pour chaque étudiant et affiche sa photo ou 0.png ;
- mémorise l'identifiant de l'étudiant dont la photo a été sélectionnée ;
- vérifie la photo à l'aide des fonctions fichierValide() et verifierImage() de la bibliothèque ficher.js;
- envoie le fichier et l'identifiant à ajax/majphoto.php au moyen d'un FormData ;
- après succès, actualise l'image affichée et rend visible le bouton de suppression.

Les contrôles JavaScript améliorent l'expérience utilisateur, mais ne remplacent pas la validation serveur : une requête peut être fabriquée sans passer par l'interface.

La validation serveur dans InputFile et ImageManager reste la référence.

5. Compléter le script index.js

Le fichier public/photo/ajax/majphoto.php réalise les actions suivantes après le chargement du bootstrap :

1. Vérification d'un appel AJAX POST sécurisé
2. Récupération des données de la requête
3. Vérification de la présence du fichier
4. Recherche de l'étudiant pour vérifier son existence et récupérer la photo actuelle
5. Instanciation d'un nouvel objet InputFileImg
6. Vérification de la validité du fichier image
7. Instanciation d'un objet FileManager
8. Sauvegarde physique de la nouvelle photo (nom d'origine, renommé si nécessaire)
9. Instanciation de l'objet métier Etudiant
10. Mise à jour de la base de données avec le nom de la nouvelle photo
11. Si la mise à jour de la BDD échoue, suppression de la photo créée sur le disque
12. Si la BDD est mise à jour avec succès et qu'il existe une ancienne photo, suppression de l'ancienne photo du disque;
13. Envoi de la réponse JSON : envoi du nom de la nouvelle photo pour mise à jour côté client

6. Compléter le script public/photo/ajax/majphoto.php
7. Vérifier le fonctionnement de la mise à jour de la photo

Le fichier public/photo/ajax/supprimerphoto.php réalise les actions suivantes après le chargement du bootstrap :

1. Vérification d'un appel AJAX POST sécurisé
2. Récupération des données de la requête
3. Recherche de l'étudiant pour vérifier son existence et récupérer la photo actuelle
4. Si la photo est null : ReponseJson::envoyerErreur("L'étudiant n'a pas de photo à supprimer.");
5. Création de l'objet métier
6. Mise à jour de la colonne photo qui prend la valeur null
7. Instanciation du FileManager pour supprimer le fichier photo de l'étudiantde réussite.
8. Suppression de la photo et envoie d'un message personnalisé en fonction des cas

La photo de l'étudiant a été supprimée de la base de données, mais le fichier n'a pas pu être retiré du disque.
La photo de l'étudiant a été supprimée de la base de données, mais le fichier n'existait pas sur le disque.
La photo de l'étudiant a été supprimée de la base de données et du disque.

8. Compléter le script public/photo/ajax/supprimerphoto.php.
9. Vérifier le fonctionnement de la suppression.

4. Modification du module 'ajout'

10. Intégrer dans le module d'ajout la possibilité d'ajouter la photo de l'étudiant.